

Real-World Malware Intelligence for Thraksha: Sourcing, Licensing, Schema, and Integration

TL;DR

- **Build Thraksha's real-intelligence corpus on abuse.ch** — MalwareBazaar (APK SHA-256 hashes + the Code Signing Certificate Blocklist), ThreatFox (Android `apk.*` family IOCs), and URLhaus (malware-distribution URLs) — because these are the only reputable free-to-query sources that cover Android specifically and expose the exact indicator types Thraksha matches. **Do NOT build on VirusTotal's public API**: its terms forbid commercial products and substitute-AV use.
- **The single biggest risk is licensing, not technology.** abuse.ch data is free to *query* under "fair use / non-commercial" terms, but its published terms are effectively silent on *redistributing* indicators to your users inside a signed ThreatPack — so bundling and shipping those indicators is a **LEGAL / LICENSING REVIEW REQUIRED** item that must be resolved in writing with Spamhaus/abuse.ch before any commercial/investor build.
- **For Android, trust APK SHA-256 and malicious signing-certificate fingerprints as strong indicators; treat package names, IPs, and URLs as weak, expiring contextual evidence.** Ship a small offline pack (a few thousand indicators, well under 1 MB) and make the demo about *scanning real apps against real intelligence and honestly reporting zero matches*, plus a controlled VillainCaller detection — not about forcing malware onto a clean phone.

Key Findings

1. **abuse.ch is the backbone.** MalwareBazaar (malware sample hashes incl. APKs), ThreatFox (multi-type IOCs incl. `apk.*` families), URLhaus (malware-distribution URLs), and SSLBL (malicious TLS certs/JA3) are all free-to-query under abuse.ch "fair use" principles, now operated in partnership with Spamhaus. They cover exactly the indicator types Thraksha inspects.
2. **abuse.ch now requires a free Auth-Key** (from `auth.abuse.ch`) for API and export access, and enforces fair-use rate limits; commercial/for-profit use, per the ThreatFox Community API page, "may require a paid subscription for the enhanced abuse.ch commercial API."
3. **ThreatFox explicitly expires IOCs older than 6 months** (since 2025-05-01) to avoid false positives, "mostly happening on cloud based infrastructure where assets (e.g. IP addresses) are changing customers quickly" — a policy Thraksha should mirror.
4. **A dedicated Code Signing Certificate Blocklist (CSCB)** exists on MalwareBazaar (SHA-256 thumbprints, issuer/subject CN, serial, `bl_reason` = malware family). This maps directly to Thraksha's signing-certificate fingerprinting capability.
5. **VirusTotal's public API is off-limits for Thraksha's use case**: it "must not be used in commercial products or services. It can not be used as a substitute for antivirus products and it can not be integrated in any project that may harm the antivirus industry directly or indirectly," with "immediate permanent ban" for non-compliance. Its Enterprise agreement also forbids sublicensing/redistribution of any Samples, datafeed, or metadata.
6. **AndroZoo (University of Luxembourg)** is the best Android-specific research corpus — "approximately 24 million APK files" per its MSR 2024 retrospective, each keyed by SHA-256 with SHA-1/MD5, market, signing cert, and per-AV labels — but its access conditions **prohibit redistribution and commercial use** and require a faculty/permanent-position sponsor. Usable for internal research/curation only, not for shipping, and likely unavailable to a solo non-academic developer.

7. **Android indicator reliability:** APK SHA-256 and malicious signer certs are strong; package names are weak (spoofable/reused); network IOCs (IP/domain/URL) decay fast and suffer from shared-hosting/CDN false positives.

Details

What Thraksha Needs From Threat Intelligence

Thraksha's scanner inspects: package identity, signing-certificate SHA-256, base-APK SHA-256, permissions/profile, and network destinations. The IOC feed therefore needs, in priority order: (1) **APK SHA-256 hashes** with malware-family labels; (2) **malicious signing-certificate fingerprints**; (3) **malicious domains / IPs / IP:port / URLs** for network-destination analysis; and (4) rich metadata (family, first/last seen, confidence, source) so PolicyEngine can reason rather than blindly quarantine. Package names are useful only in combination with a hash or signer.

Source Comparison Table

Source	Android relevance	SHA256	Certs	Pkg names	Domains	IPs	URLs	Fa
MalwareBazaar (abuse.ch)	High (apk tag/signature)	✓	✓ (CSCB)	partial	–	–	–	✓
ThreatFox (abuse.ch)	High (apk.* families, Android tag)	✓	–	via context	✓	✓	✓ (ip:port)	✓
URLhaus (abuse.ch)	Medium (mobile malware URLs)	✓ (payload)	–	–	✓	✓	✓	✓
SSLBL (abuse.ch)	Low–Med (C2 TLS)	–	✓ (SHA-1)	–	–	✓	–	✓
AndroZoo (uni.lu)	Very High	✓	✓ (metadata)	✓	–	–	–	vital
VirusTotal	High	✓	✓	✓	✓	✓	✓	✓
AlienVault OTX	Medium	✓	–	–	✓	✓	✓	vital
OpenPhish (Community)	Low (phishing URLs)	–	–	–	✓	–	✓	–
CISA AIS	Low	✓	–	–	✓	✓	✓	–

Detailed Review of Each Source

MalwareBazaar (abuse.ch/Spamhaus). Purpose: sharing confirmed malware samples ("over 1'120'404 confirmed malware samples" per the FAQ; "Over 500 daily consumers"; "around 100 million API requests in 30 days," Oct 2024). Provides SHA-256/SHA-1/MD5, imphash/tlsh/telfhash/gimphash, signature (family), tags (including `apk / android`), `first_seen`, and a **Code Signing Certificate Blocklist (CSCB)** – code-signing certs dumped by ReversingLabs' A1000 platform and manually vetted by abuse.ch – with SHA-256 thumbprint, issuer/subject CN, serial number, validity, and `cscb_reason / bl_reason` (family, e.g. "CobaltStrike," "Quakbot"). Access: REST API (`mb-api.abuse.ch`), bulk CSV/hash exports, hourly/daily batches, and daily MISP events. **Auth: free Auth-Key required.** The file-download API is limited to 2,000 samples/IP/day (irrelevant to Thraksha, which never downloads binaries). Update cadence: recent exports every 5 min, full dumps hourly. [MalwareBazaar](https://malwarebazaar.abuse.ch/) Quality is high (confirmed malware only; no adware/PUP). [MalwareBazaar](https://malwarebazaar.abuse.ch/) For Thraksha, use the **hash-list exports and the CSCB CSV** (`/export/csv/cscb/`) – metadata only, never the sample ZIPs (password `infected`).

ThreatFox (abuse.ch/Spamhaus). Purpose: sharing IOCs associated with malware ("over 1'808'084 IOCs"). IOC types include `sha256_hash`, `domain`, `ip:port`, `url`, and more; each carries a malware family (including Android families `apk.anatsa` [aka ReBot/TeaBot/Toddler], `apk.hydra`, `apk.888_rat`, `apk.irata`, `apk.smspy`, `apk.doublelocker`), a submitter-chosen confidence level (abuse.ch warns to "only use high confidence levels on IOCs you have manually vetted"), `first/last seen`, and `reporter/reference`. There is an "Android" tag. Access: REST API (`threatfox-api.abuse.ch/api/v1/`), data export (CSV/JSON), host-file and Suricata/RPZ formats, and a MISP event feed. **Auth: free Auth-Key. IOCs older than 6 months are expired** and dropped from API/export (still visible in UI). [Abuse](https://threatfox.abuse.ch/) No STIX/TAXII – abuse.ch dropped it because the export grew to gigabytes. This is Thraksha's richest single source for Android network + hash IOCs with family labels.

URLhaus (abuse.ch/Spamhaus). Malicious URLs used for malware distribution, [URLhaus](https://urlhaus.abuse.ch/) with associated payload hashes (incl. SHA-256), host, status (online/offline), tags, and timestamps ("Over 5000 daily consumers"). Access: REST API, full/active/recent database dumps (CSV/JSON, every 5 min), and MISP daily feed. Note the ISP/CERT country/ASN/TLD feeds are explicitly "not intended for blocklisting... nor for being used as an Indicator Of Compromise" – use the API datasets, not those feeds. URLs disappear fast; treat as short-lived.

SSLBL (abuse.ch). Malicious SSL certificates (SHA-1 fingerprints) and JA3 fingerprints tied to botnet C2, plus a botnet C2 IP blocklist. abuse.ch itself warns the JA3 list is "not tested against known good traffic yet and may cause a significant amount of FPs," and the IP RPZ contains only IPs seen in the past 30 days. Low direct Android relevance; optional network enrichment only.

AndroZoo (**University of Luxembourg**). The premier academic Android corpus – "approximately 24 million APK files" (MSR 2024 retrospective) from Google Play, F-Droid, Anzhi (closed 2023), AppChina, etc., each keyed by SHA-256 with SHA-1/MD5, size, market, signing certificate, and per-AV detection labels. Access: email application (`androzoo@uni.lu`) with an institution/faculty sponsor; an API key capped at **500,000 APK downloads per 6-month key**. **Licensing is restrictive:** the maintainers "request that researchers... (2) Do not, in general, redistribute the data; (3) Do not, in particular, make a commercial usage of this data" and require "a faculty, or someone in a permanent position, to agree and commit to those conditions." Consequence for Thraksha: excellent for *internally validating* which APK hashes/certs correspond to real-world malware and for building label ground-truth, but the metadata **cannot be redistributed or used commercially** and so cannot feed the shipped ThreatPack. As a solo developer in England without a faculty sponsor, Thraksha may not even qualify – **LEGAL / LICENSING REVIEW REQUIRED**, and simplest to avoid.

VirusTotal. Broadest coverage and best enrichment, but the public API "must not be used in commercial products or services. It can not be used as a substitute for antivirus products and it can not be integrated in any project that may harm the antivirus industry directly or indirectly," with "immediate permanent ban" for non-compliance; users may not redistribute Samples; and partner AV attributions may not be shown publicly without permission. The Enterprise agreement forbids sublicensing/distributing "any Samples, datafeed, metadata or results... to any third party." [VirusTotal](https://www.virustotal.com/) **Verdict: unusable for Thraksha's shipped product; even as**

a backend enrichment source it would require a paid VT Enterprise license and its no-redistribution clause would still bar embedding VT data in the pack (VT could inform internal triage only).

AlienVault OTX (now LevelBlue). Community pulses (IOCs of many types) via the DirectConnect REST API, exportable as CSV/OpenIOC/STIX. "The DirectConnect API is free to OTX users for any non-commercial use. Commercial partners use the API under the conditions described in a license agreement." [Manualzz](#) Quality is variable (crowd-sourced, duplicated, sometimes stale). Useful as a **secondary corroborating source** in the backend, not primary or shippable; commercial use requires a license.

OpenPhish. The Community feed is a subset of phishing URLs refreshed every 12 hours; the Terms of Use state: "You agree not to use any part of the Services for any commercial purposes without the prior written consent of OpenPhish." [OpenPhish](#) Commercial redistribution requires Premium/Platinum. Low Android relevance; skip for now.

CISA AIS. Free (no-cost) US government STIX 2.1 / TAXII 2.1 exchange of cyber threat indicators; requires a DHS-approved medium-assurance PKI certificate and agreement to a Terms of Use, and is oriented to organizations/ISACs sharing bidirectionally. Heavy onboarding for a solo UK developer and limited Android value; not recommended now.

MISP. Not a source but a *format/platform*. Several abuse.ch feeds ship as MISP events, and MISP ships default community feeds (e.g., CIRCL OSINT). Relevant only for the future backend as a normalization/aggregation hub.

Best Sources for Android Malware

1. **MalwareBazaar** — APK SHA-256 + CSCB signing certs (strongest Android hits).
2. **ThreatFox** — `apk.*` families with hashes and network IOCs, family-labeled.
3. **AndroZoo** — internal ground-truth/validation only (no shipping).

Best Sources for Network IOCs

1. **ThreatFox** — domains, IPs, ip:port, URLs with family + confidence + 6-month expiry.
2. **URLhaus** — active malware-distribution URLs with online/offline status.
3. **SSLBL / Feodo Tracker** — C2 IPs and TLS certs (optional enrichment).

Licensing / Redistribution Analysis

The locked constraint — an **offline signed ThreatPack embedded in the APK, redistributed to every user, in a product intended for an investor demo and eventual commercial sale** — is exactly the case where "free to query" ≠ "free to redistribute." Applying the A–G test:

- **abuse.ch (MalwareBazaar/ThreatFox/URLhaus/SSLBL):**
 - A. Query privately — **Yes**, under fair use with a free Auth-Key.
 - B. Store internally — **Yes** (implied by fair-use querying).
 - C. Normalize into Thraksha's schema — **Not addressed** in the published terms → **REVIEW REQUIRED** (likely acceptable as internal processing).
 - D. Embed a subset in an offline demo pack — **Not explicitly permitted** → **LEGAL / LICENSING REVIEW REQUIRED**.
 - E. Redistribute indicators to users — **Not explicitly permitted** → **LEGAL / LICENSING REVIEW REQUIRED**. The reachable abuse.ch terms speak only to *query volume* and *commercial vs. non-commercial* use, and are silent on redistribution to third parties/end users.
 - F. Commercial product use — abuse.ch states commercial/for-profit use "may require a paid subscription" via Spamhaus → for anything beyond a personal/non-profit demo, **obtain the Spamhaus commercial arrangement in writing**.

- G. Display source attribution/classification to users — **Not prohibited; advisable as good practice.**
No formal attribution clause was found, but showing "Source: abuse.ch ThreatFox" is recommended.
 - **Verdict:** Fine to build and demo *privately* now; before you ship indicators to users or raise money on the capability, **email Spamhaus/abuse.ch and get written confirmation** of (a) redistribution of a small embedded subset and (b) commercial use. This is a gating legal task, not a formality.
- **AndroZoo:** A–B internal research only; **C/D/E/F prohibited** (no redistribution, no commercial use); access may be unavailable to a solo developer. Internal validation only, if at all.
- **VirusTotal:** A permitted under the public API's non-commercial ToS *only*; **B/C internal permitted but D/E/F/G prohibited** for a commercial substitute-AV product. Do not build on VT.
- **AlienVault OTX:** A–C free for non-commercial; **F requires a commercial license**; D/E ambiguous → REVIEW REQUIRED.
- **OpenPhish Community:** commercial use requires written consent; **do not redistribute.**
- **CISA AIS:** governed by its ToU/MISA; onboarding-heavy; not recommended for this product.

Bottom line: only abuse.ch realistically supports the shipped-pack model, and only after a written redistribution/commercial arrangement with Spamhaus. Until then, the honest investor framing is "Thraksha queries and is validated against real abuse.ch intelligence," with the embedded pack labeled as a curated subset pending the commercial agreement.

Android Indicator Reliability Analysis

APK SHA-256 — STRONG but brittle. An exact base-APK SHA-256 match is high-confidence proof that a specific known-bad artifact is present, with essentially zero false positives. The weakness is coverage: Android malware is heavily *repackaged* and rebuilt, so any code change, re-signing, or resource tweak yields a completely different hash (a large body of research confirms repackaging is the dominant malware-distribution technique). Split APKs (App Bundles) mean the base-APK hash differs from what a store served. **Recommendation:** trust an APK SHA-256 match as a "KNOWN THREAT MATCH," but never rely on hashes alone for coverage — pair with signer-cert matching.

Signing-certificate fingerprints — STRONG for campaigns. Because repackaged/rebuilt malware within one campaign is frequently signed by the *same* threat-actor key, a malicious signer fingerprint catches many hash variants at once — Thraksha's highest-leverage Android indicator. Caveats: (1) *stolen/leaked platform certificates* exist — the Nov 2022 Android Partner Vulnerability Initiative (APVI) disclosure by Google reverse engineer Łukasz Siewierski showed platform certificates belonging to Samsung, LG, Revoview, and Mediatek being used to sign Android malware (Google advised affected vendors to "rotate the platform certificate by replacing it with a new set of public and private keys"), so a compromised cert can appear on both good and bad apps — match on the *malware-associated* cert, not the vendor's legitimate current cert; (2) *shared debug/test keys* (the public AOSP test keys) cause false positives — maintain an allowlist of well-known debug/test certificate fingerprints and never quarantine on those; (3) Android v2/v3 signing and key rotation (SigningInfo signing history) mean an app can present multiple certs — evaluate the full signing lineage. **Recommendation:** treat a vetted malicious-signer match as strong evidence, second only to a hash match, guarded by a debug/test-key allowlist.

Package names — WEAK. Package IDs are trivially spoofable and reusable; benign and malicious apps can share names, and malware rotates package names freely. A package-name match alone must never trigger containment. **Recommendation:** use it only as a corroborating signal *with* a signer or hash match, or to raise "SUSPICIOUS / REVIEW."

Domains / IPs / URLs — WEAK and time-decaying. Fast-flux, cloud, CDN, and shared hosting mean a malicious IP today is a benign SaaS tenant next week; domains change ownership; URLs vanish. abuse.ch's own 6-month IOC expiry and 30-day IP RPZ windows codify this. **Recommendation:** use network IOCs only for

the network-destination *analysis* (contextual evidence / "HIGH-CONFIDENCE IOC MATCH" at best when fresh and corroborated), with aggressive expiry.

Recommended Thraksha IOC Schema

Normalized record (JSON canonical form; stored on-device in a compact binary/SQLite form):

indicatorId	(string, UUID)	– MANDATORY
indicatorType	(enum)	– MANDATORY
indicatorValue	(string, canonicalized)	– MANDATORY
platform	(enum: android windows cross network)	– MANDATORY
classification	(enum: malware c2 distribution phishing pua suspicious)	– MANDATORY
malwareFamily	(string, normalized, e.g. "anatsa")	– optional
severity	(enum: low medium high critical)	– MANDATORY
confidence	(int 0-100, source-declared)	– MANDATORY
source	(enum: malwarebazaar threatfox urlhaus sslb internal)	– MANDATORY
sourceReference	(string/URL to origin record)	– MANDATORY (provenance)
firstSeen	(ISO-8601 UTC)	– optional
lastSeen	(ISO-8601 UTC)	– optional
expiresAt	(ISO-8601 UTC)	– MANDATORY
revoked	(bool, default false)	– MANDATORY
enabled	(bool, default true)	– MANDATORY
tags	(array<string>)	– optional
license	(enum/string, e.g. "abuse.ch-fairuse")	– MANDATORY
attribution	(string, display credit)	– optional
metadata	(object, free-form)	– optional

Enums for `indicatorType`: `apk_sha256`, `cert_sha256`, `cert_sha1`, `package_name`, `domain`, `ipv4`, `ipv6`, `ip_port`, `url`, `ja3`.

Validation & canonicalization rules:

- **SHA-256**: lowercase hex, exactly 64 chars, `^[0-9a-f]{64}$`. Normalize uppercase.
- **Certificate fingerprint**: lowercase hex, no colons/separators, algorithm recorded separately (`cert_sha256` = 64 hex; `cert_sha1` = 40 hex). Store issuer/subject CN and serial in `metadata`.
- **Domain**: lowercase, IDNA/punycode-normalized, trailing dot stripped, no scheme.
- **IPv4/IPv6**: normalized (IPv6 lowercased, zero-compressed); `ip_port` stored as `ip|port`.
- **URL**: scheme+host lowercased, host punycode-normalized, fragment stripped; path/query kept verbatim.
- **package_name**: validated against the Android package regex; always low-trust.
- **Deduplication**: natural key = (`indicatorType`, canonical `indicatorValue`). On collision, merge sources into a `sources[]` list and keep max severity + max confidence + earliest firstSeen + latest lastSeen. This is what enables corroboration counting.

Confidence / Source Trust Model

Do not present confidence as an ML probability. Use a deterministic, explainable tiering:

- **Source trust level** (Thraksha-assigned): MalwareBazaar/ThreatFox = High; URLhaus = High for URLs; SSLBL/JA3 = Medium (FP-prone); OTX = Medium; internal demo indicators = labeled Demo.
- **Indicator-type strength**: `apk_sha256` & `cert_sha256` = Strong; `package_name` = Weak; domain/ip/url = Weak-and-decaying.
- **Corroboration count**: independent sources asserting the indicator.
- **Freshness**: age vs. `expiresAt`.
- **Contextual findings**: Thraksha's own permission/profile analysis and network evidence.

Decision bands (feed PolicyEngine, don't replace it):

- **KNOWN THREAT MATCH** → exact `apk_sha256` match to a High-trust, non-expired indicator, OR a vetted malicious `cert_sha256` match not on the debug/test allowlist. Eligible for Device Owner containment (subject to Advice Mode / user policy).
- **HIGH-CONFIDENCE IOC MATCH** → a strong network IOC that is fresh AND corroborated by ≥2 sources, OR reinforced by Thraksha's own contextual/network evidence. Warn/recommend, don't auto-quarantine.
- **SUSPICIOUS / REVIEW** → weak single-source match (package name, stale IP, low submitter confidence) or contextual heuristics only. Surface for user/analyst review.

This structurally prevents the "one random feed says bad → auto-quarantine" failure: containment requires a Strong indicator type from a High-trust source.

Staleness / Expiration Policy

Set `expiresAt` at ingestion by type:

- **apk_sha256**: long-lived — 24 months or non-expiring (historical artifacts stay valid); re-affirm on re-sighting.
- **cert_sha256 (malicious signer)**: long-lived — 12–24 months (campaign keys persist), but auto-revoke immediately if the cert is a known-leaked *platform* cert that vendors have rotated.
- **domain**: 3–6 months.
- **ipv4/ipv6/ip_port**: 30–90 days (align with abuse.ch's 30-day IP RPZ / 6-month IOC window; prefer the shorter end).
- **url**: 30–90 days, and downgrade when URLhaus marks it offline.
- **package_name**: never authoritative; 6 months and only as corroboration.
Expired indicators are disabled for matching (`enabled=false`) but retained for audit/history.

Safe Investor Demo Dataset

Composition (metadata/IOCs only, no binaries), a small curated pack:

- 1,500–3,000 **APK SHA-256** hashes (MalwareBazaar `apk` signature/tag exports, family-labeled).
- 200–500 **malicious signing-certificate fingerprints** (CSCB + Android-relevant certs).
- 1,000–2,000 **domains/IPs/URLs** from ThreatFox `apk.*` families + URLhaus (fresh, non-expired).
- 100–300 **package-name** corroboration entries (low-trust, tagged).
- 1 clearly-labeled **VillainCaller** controlled indicator for the deterministic detection demo.
Total on the order of a few thousand indicators — well under 1 MB in compact form.

Why this is defensible: every indicator is traceable to a named reputable source (`sourceReference`), none is fabricated, no malware binary is present, and the scanner logic is identical whether it hits demo or real data. The truthful claim "Thraksha is scanning real installed apps against real-world threat intelligence from abuse.ch" holds because the corpus is genuinely sourced, the hashes/certs are real, and the match engine is the production engine.

Demo choreography (per the locked constraint):

1. Scan real installed apps on the S20 FE / Z Flip4.
2. Compute real base-APK SHA-256 and signer cert SHA-256; compare against the corpus.
3. **Honestly report zero known matches** if the device is clean — demonstrating integrity and a working negative path.
4. Separately trigger the controlled **VillainCaller** indicator to show a KNOWN THREAT MATCH → PolicyEngine → Advice/containment flow end-to-end.
5. Show the audit log proving the same engine used the real corpus for both.
This is more credible than planting malware because investors see an honest true-negative (the hard part any real scanner must get right) *and* a controlled true-positive, with no theatrics or fabricated infection.

Production Intelligence Pipeline (FUTURE PHASE — design only)

External sources (abuse.ch APIs/exports, licensed feeds) → ingestion (scheduled pull, per-source rate limits + Auth-Key) → provenance validation (record source, signature, fetch time) → schema normalization → deduplication/merge (natural key, source list) → confidence calculation (trust × type × corroboration × freshness) → expiration/revocation (per-type TTL, kill list) → Thraksha intelligence DB (server-side; PostgreSQL/MISP) → compact mobile ThreatPack generation (subset, binary/SQLite) → cryptographic signing → secure distribution (HTTPS + pinning) → device signature verification → atomic activation → rollback to last known-good → audit update .

STIX/TAXII vs MISP on the backend: MISP is the pragmatic hub — abuse.ch already publishes MISP feeds, it handles dedup/correlation/expiry, and it exports normalized data. STIX/TAXII is worth supporting only if you later ingest CISA AIS or enterprise partners; note abuse.ch deliberately dropped STIX/TAXII for size reasons. [Abuse](#) The mobile device should NOT receive STIX or MISP JSON — it should receive Thraksha's compact signed format. Keep the rich standards on the backend; ship a lean derived pack.

Secure Update / Signing Architecture (FUTURE PHASE — design only)

Thraksha already verifies RSA-signed packs. Harden the supply chain with:

- **Two-tier key hierarchy:** an **offline root key** (air-gapped, signs a rotating signing key/certificate) and an **online signing key** (signs each pack). Compromise of the online key is recoverable by root-signing a new one and revoking the old.
- **Version monotonicity:** each pack carries a strictly increasing version + build timestamp; the device rejects any pack with version ≤ the active version (**rollback prevention**).
- **Expiry:** packs carry a `notAfter` ; an expired pack triggers a fetch and, if offline too long, a degraded "stale intelligence" state rather than silent trust.
- **Emergency revocation / kill switch:** a signed revocation list can disable specific `indicatorId`s or an entire bad pack version; the device honors it before matching (defends against poisoned indicators and mass false-positives).
- **Staged activation:** verify signature → verify version monotonicity → verify expiry → stage → atomically switch → keep the previous pack for one-tap rollback.
- **Provenance in-pack:** each indicator retains `source` / `sourceReference` so a bad source can be surgically purged.
- **Anti-mass-quarantine safeguard:** cap the number of *new* containment-eligible matches a single pack update may trigger without user confirmation; require the KNOWN-THREAT tier for any automated containment.

Performance / On-Device Storage Strategy

Targets: Galaxy S20 FE (6 GB RAM, Snapdragon 865, Android 13 — the floor) and Z Flip4 (8 GB, SD 8+ Gen 1). For a few-thousand-indicator demo pack and even a production pack of low-hundreds-of-thousands of indicators, all approaches are cheap; choose by access pattern:

- **JSON:** fine for the tiny signed demo pack (human-auditable, trivially parsed). Poor for large sets (parse time, memory).
- **SQLite:** recommended production on-device store — indexed exact-match lookups on 64-char hex are effectively constant-time at these sizes, support incremental updates, use low RAM, and are native to Android.
- **Bloom filter:** ideal *front filter* for hash/cert set-membership — a compact bit array gives sub-microsecond "definitely not present" answers with no false negatives, backed by SQLite for the definitive check on a possible hit. (Forensic tooling reports dropping hash-DB lookup time from ~55 s to ~0 s by adding a Bloom pre-filter.) This keeps per-app scan cost negligible even for hundreds of thousands of hashes.

- **Hash set/index (in-memory):** fine for the small demo pack loaded once at startup.
 - **Binary format / segmented packs:** for production, ship a compact binary + a Bloom filter, segmented by indicator type so short-TTL IP/URL segments can update independently of long-lived hash/cert segments (smaller incremental downloads).
- Recommended path:** demo = signed JSON (or SQLite) + in-memory index; production = **Bloom filter front + SQLite backing store, segmented, incrementally updated**. Do not prematurely optimize the demo.

Risks and Failure Modes

- **Licensing/redistribution overreach** (shipping abuse.ch data commercially without a Spamhaus arrangement) — the top risk; gate the commercial build on written permission.
- **Compromised/poisoned feed** — mitigated by source trust tiers, corroboration requirements, and the kill switch.
- **Mass false-positive quarantine** — mitigated by KNOWN-THREAT-tier-only containment, the debug/test-key allowlist, and per-update containment caps.
- **Stale intelligence** — mitigated by per-type expiry and a "stale pack" degraded state.
- **MITM / forged / rolled-back pack** — mitigated by RSA signing, TLS + pinning, version monotonicity, and rollback prevention.
- **Over-claiming to investors** — mitigated by the honest true-negative demo and clear Demo-vs-real labeling.

Recommended Phase-10 Assets

1. MalwareBazaar `apk / android` hash exports + full CSCB (metadata only).
2. ThreatFox `apk.*` family IOCs + Android tag (hashes, domains, ip:port, URLs, confidence).
3. URLhaus recent/active URL dump (fresh subset).
4. A curated, signed, expiry-tagged Thraksha pack built from the above.
5. Debug/test-key allowlist (AOSP public keys).
6. One controlled VillainCaller indicator.

Exact Download / API Acquisition Plan

1. Register at `auth.abuse.ch` for a free Auth-Key [GitHub](#) (one key works across MalwareBazaar/ThreatFox/URLhaus).
2. **MalwareBazaar:** pull hash exports (`/export/`) and the CSCB CSV (`/export/csv/cscb/`), and query by tag `apk/signature apk` via `mb-api.abuse.ch`. Metadata/hashes only — never fetch sample ZIPs.
3. **ThreatFox:** POST to `threatfox-api.abuse.ch/api/v1/` for `malware_list`, then pull IOCs for each `apk.*` family and the `Android` tag, or use the data export. Respect the 5-minute cadence and 6-month expiry.
4. **URLhaus:** pull the recent/active database dumps (CSV/JSON) every ≥5 minutes as needed.
5. Normalize → dedup → assign confidence/expiry → curate the subset → sign the pack.
6. **Before any commercial/investor distribution:** email Spamhaus/abuse.ch to confirm redistribution of an embedded subset and commercial terms in writing.

What NOT to Download

- **No malware binaries** (MalwareBazaar/AndroZoo sample files, vx-underground, GitHub malware-sample repos). Metadata/IOCs only.
- **No VirusTotal Samples or metadata** into the product.
- **No AndroZoo APKs or redistributable metadata** into the shipped pack.
- **No live C2 payloads, exploit code, or execution** of any kind.

Recommendations

Stack A — FREE / INVESTOR DEMO (now, non-commercial):

- **Sources:** MalwareBazaar (APK SHA-256 + CSCB), ThreatFox (`apk.*` + Android tag), URLhaus (fresh URLs).
- **Indicator types:** apk_sha256, cert_sha256, domain/ip/ip_port/url, low-trust package_name.
- **Acquisition:** free Auth-Key; REST + bulk exports; metadata only.
- **Licensing caveat:** fair-use/non-commercial only; embedding = **REVIEW REQUIRED** — label the pack "curated real-world subset."
- **Update cadence:** manual/periodic re-curation (weekly is ample for a demo).
- **Redistribution:** treat as not-yet-permitted; do not assert a commercial redistribution right.
- **Placement:** on-device signed pack.
- **Action now:** register the Auth-Key; build the few-thousand-indicator signed pack; run the honest scan → zero-match → VillainCaller-detection demo.

Stack B — EARLY PRODUCTION (before real user testing):

- **Gating benchmark:** obtain **written** redistribution + commercial-use confirmation from Spamhaus/abuse.ch.
- **Sources:** abuse.ch (same) + AlienVault OTX as backend corroboration (non-commercial or licensed).
- **Acquisition:** backend MISP-based pipeline; automated pulls with expiry/dedup/confidence.
- **Licensing caveat:** honor the abuse.ch arrangement; keep OTX non-commercial or licensed.
- **Update cadence:** backend daily; device packs weekly (segmented).
- **Redistribution:** only per the written abuse.ch agreement.
- **Placement:** rich data backend-only; lean signed pack on-device; add two-tier signing keys, version monotonicity, expiry, and kill switch.

Stack C — COMMERCIAL SCALE:

- **Sources:** licensed **Spamhaus commercial abuse.ch API / real-time feeds** (unified query, false-positive remediation, SLAs), plus optionally a paid commercial mobile-threat-intelligence feed and, if enterprise sharing is needed, CISA AIS via TAXII.
- **Indicator types:** full set with enriched metadata.
- **Acquisition:** paid API/feeds; STIX/TAXII/MISP on the backend.
- **Licensing caveat:** commercial license fees; respect any no-redistribution clauses (e.g., if VT Enterprise is ever used, its data stays internal).
- **Update cadence:** near-real-time backend; incremental segmented device packs.
- **Redistribution:** per each paid contract.
- **Placement:** backend ingestion/normalization; compact signed packs to devices.

Benchmarks that change the plan: if abuse.ch/Spamhaus declines redistribution → pivot to a commercial mobile-threat-feed vendor or generate your own indicators from AndroZoo-validated internal analysis (research-only caveats permitting). If false-positive complaints exceed a small threshold in testing → tighten to hash+cert-only containment and lengthen corroboration requirements. If pack size or scan latency becomes noticeable on the S20 FE → move from JSON to Bloom+SQLite segmented packs.

Caveats

- **abuse.ch redistribution/attribution/derivative terms are effectively silent in the published Terms of Use** — the reachable terms address query volume and commercial-vs-non-commercial use, not third-party redistribution or derivative/normalized indicators. Every "embed/ship/commercial" step is therefore

- LEGAL / LICENSING REVIEW REQUIRED** and must be confirmed in writing with Spamhaus/abuse.ch.
This is verified-as-ambiguous, not verified-as-permitted. (The full abuse.ch Terms-of-Use page could not be loaded programmatically; a human should read <https://abuse.ch/terms-of-use/> in full before the commercial build.)
- The 6-month ThreatFox expiry, 2,000/day MalwareBazaar download cap, free-Auth-Key requirement, and CSCB structure are verified from official abuse.ch docs (current as of 2026).
 - VirusTotal, AndroZoo, OpenPhish, OTX, and CISA AIS restrictions are quoted from their own documentation/terms; treat AndroZoo access eligibility for a solo non-academic developer as uncertain.
 - Confidence values from crowd-sourced feeds (ThreatFox, OTX) are submitter-declared, not independently validated — hence the trust-tier model.
 - The Samsung/LG/Mediatek platform-certificate abuse is verified via the Nov 2022 APVI disclosure (Łukasz Siewierski / Google) as reported by BleepingComputer; it illustrates why a cert allowlist and "malware-associated cert" matching are required.
 - Performance figures are directional (based on the nature of hash-set/Bloom/SQLite lookups); benchmark on the actual S20 FE before finalizing the production format.